

ENCN205

Applied Data Analysis for Civil and Natural Systems

LECTURE 8

Raster Analysis

University of Canterbury | Te Whare Wānanga o Waitaha

Dr Alberto Ardid

Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |
alberto.ardid@canterbury.ac.nz

Rasters are the language of continuous phenomena.

Elevation changes smoothly across the landscape

Temperature varies continuously, not in sharp boundaries

Rainfall does not stop at a property line

Satellite imagery captures every square metre

Raster grids let us model, measure, and analyse these fields computationally.

Raster Properties

Resolution — cell size (e.g. 1 m, 10 m, 30 m) determines detail

Extent — geographic bounding box (xmin, ymin, xmax, ymax)

CRS — coordinate reference system (must match other layers!)

Bands — number of data layers (DEM = 1 band, RGB imagery = 3 bands)

NoData — sentinel value for missing/invalid cells (e.g. -9999, NaN)

Data type — int8, int16, float32 — affects precision and file size

Always inspect these properties before analysis: `raster.rio.crs`, `raster.rio.resolution()`

Always ask: 'what resolution — and what is NoData?'

Reading Rasters in Python

```
import rioarray
```

```
dem = rioarray.open_rasterio('canterbury_dem.tif')
```

Key attributes:

```
dem.shape      # (bands, rows, cols)
```

```
dem.rio.crs     # coordinate reference system
```

```
dem.rio.resolution() # (x_res, y_res) in CRS units
```

```
dem.rio.bounds() # (left, bottom, right, top)
```

```
dem.rio.nodata  # NoData sentinel value
```

Rioarray wraps xarray + rasterio — gives you labelled arrays with CRS awareness.

Always ask: 'what CRS is this raster in?'

Map Algebra — Four Types of Raster Operations

Map algebra lets us perform mathematical or analytical operations on raster cells.



1 Local: Cell-by-Cell Arithmetic

Operations are applied to cells in the same position.

Raster A				Raster B				A + B		
2	4	1		1	2	3		3	6	4
3	5	2	+	2	1	2	=	5	6	4
1	3	4		3	2	1		4	5	5

i Examples: +, −, ×, ÷, power, min, max, etc.



2 Focal: Moving Window (Neighbourhood)

Calculations use a window of neighbouring cells.

4	2	7	1	3
5	8	3	6	2
1	4	9	2	5
3	6	1	7	4
2	5	3	8	1

3×3 window
mean = 4.7

$$(8 + 3 + 6 + 4 + 9 + 2 + 6 + 1 + 7) / 9 = 4.7$$



Examples: mean, sum, min, max, range, focal statistics, filters, etc.



3 Zonal: Statistics by Region

Calculate statistics for cells within zones.

10	15	12	8	6
14	11	9	7	5
13	20	18	10	8
22	19	17	15	9
25	21	16	14	11

Zone 1 (green)
mean = 12.5
(7 cells)

Zone 2 (orange)
mean = 7.8
(8 cells)

Zone 3 (purple)
mean = 17.8
(10 cells)

i Useful for comparing areas (e.g., by land use, watersheds, districts).



4 Global: Whole-Raster Operation

Uses the entire raster to create a new output.

2.8	2.2	2.0	2.2	2.8
2.2	1.4	1.0	1.4	2.2
2.0	1.0	SRC	1.0	2.0
2.2	1.4	1.0	1.4	2.2
2.8	2.2	2.0	2.2	2.8

Distance from source cell (SRC)

Each cell shows the shortest distance (in cell steps) from the source.



Examples: distance, cost distance, viewshed, flow accumulation, etc.



Key takeaway: All four operations transform rasters in different ways — locally, neighbourhood-based, by zones, or across the whole raster.

Map algebra is the mathematical framework for raster computation — invented by Dana Tomlin (1983).

Local Operations — Cell-by-Cell

Each output cell = function of input cell(s) at the same position

Arithmetic:

```
ndvi = (nir - red) / (nir + red)  # vegetation index
```

```
flood_depth = water_level - dem  # depth map
```

Reclassify:

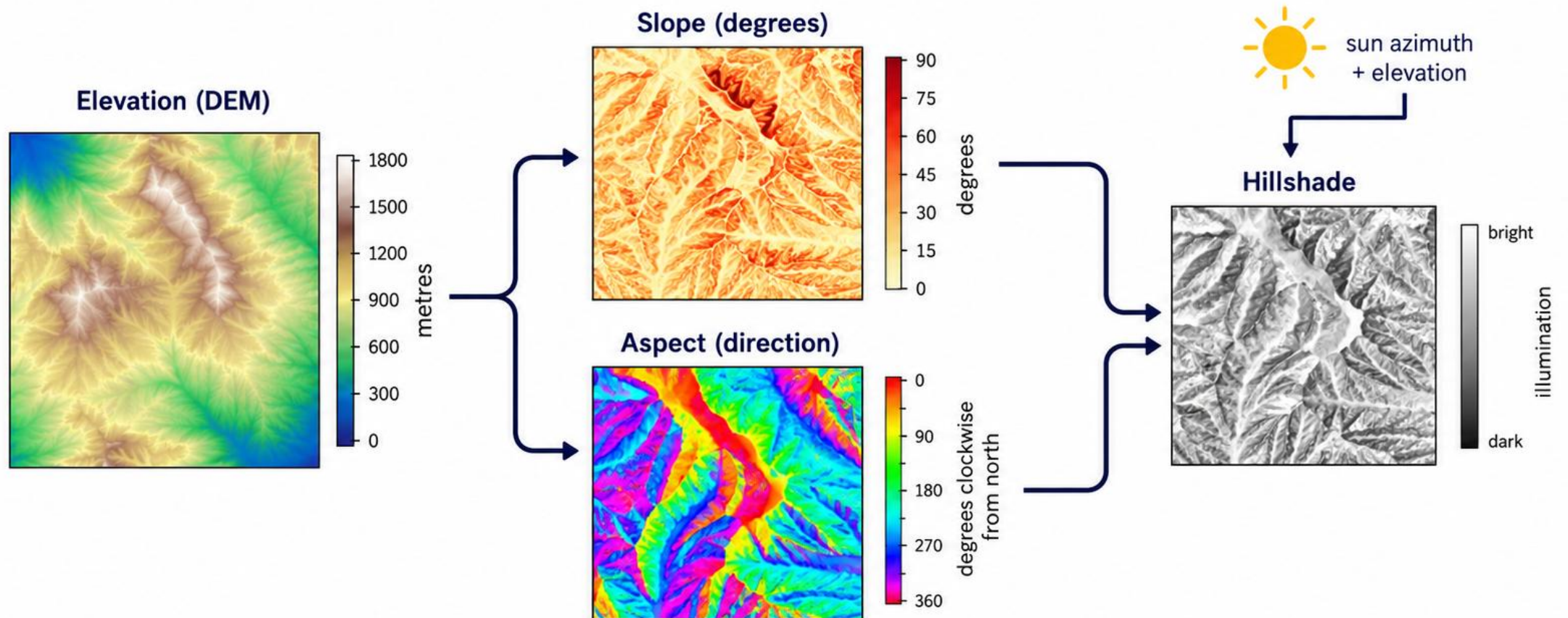
```
slope_class = xr.where(slope > 15, 'steep', 'gentle')
```

Mask:

```
masked = dem.where(dem > 0)  # remove ocean cells
```

In NumPy/xarray, these are just array operations — fast and vectorised.

Focal Operations - Terrain Products (derived from DEM)



Slope and aspect are **independently** derived from the DEM using a local 3x3 neighbourhood.

Hillshade combines slope, aspect, and sun position to simulate illumination.

Aspect is undefined on perfectly flat terrain.

Zonal Operations — Statistics by Region

Summarise raster values within defined zones (regions)

Common zonal statistics:

Mean elevation per catchment

Maximum slope per land parcel

Total rainfall per district

Area of each land cover class within a region

Implementation:

Zones can be another raster (land use classes) or vector polygons

In Python: use `rasterstats.zonal_stats()` or manual masking + NumPy

Zonal statistics bridge raster analysis and vector reporting.

Global Operations — Whole-Raster Computation

Every cell in the output depends on the entire input raster

Distance surfaces:

Euclidean distance from rivers, roads, or hazard sources

Cost-distance: weighted by terrain difficulty or land cover

Viewshed:

Which cells are visible from a given observation point?

Flow accumulation (next lecture):

How many upstream cells drain through each cell?

Global operations are computationally expensive — they scan the entire raster.

LIVE DEMO

[L08_RasterAnalysis.html](#)

Adjust sun azimuth and altitude interactively to see how hillshade reveals terrain structure invisible in raw elevation data.

Scan to open the demo on your phone



aardid.github.io/uc_205_gis_course/L08_RasterAnalysis.html

Terrain Analysis Workflow

Step-by-step: from raw DEM to engineering product

1. Load DEM and inspect properties (CRS, resolution, NoData)
2. Compute slope from DEM (focal: 3x3 window)
3. Classify slope into engineering categories:
0-5° = flat | 5-15° = moderate | 15-30° = steep | >30° = very steep
4. Overlay with land use or infrastructure for risk assessment
5. Generate hillshade for cartographic presentation
6. Map: classified slope + hillshade background + vector overlays

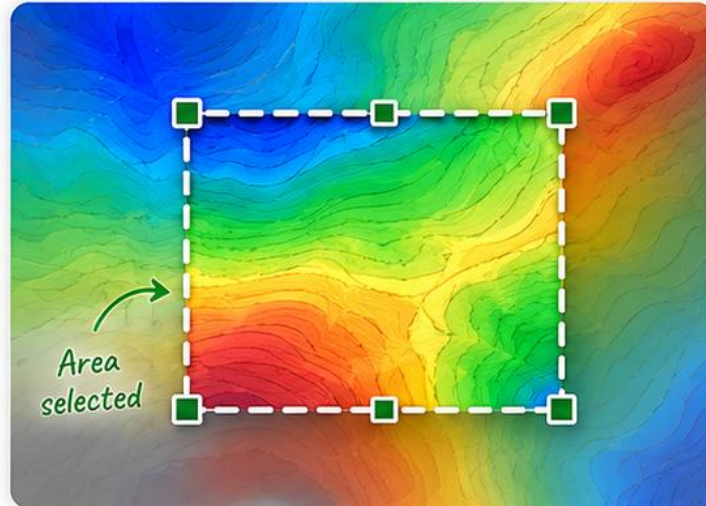
This workflow is the template for terrain-based engineering assessments.

Raster-Vector Interaction

Vector geometries are used to select or retrieve values from raster data.

1 Crop

Clip raster to a bounding box.

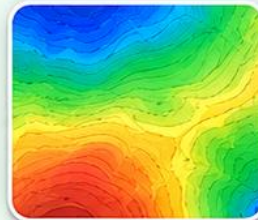


What it does

Returns only the raster cells inside the bounding box.

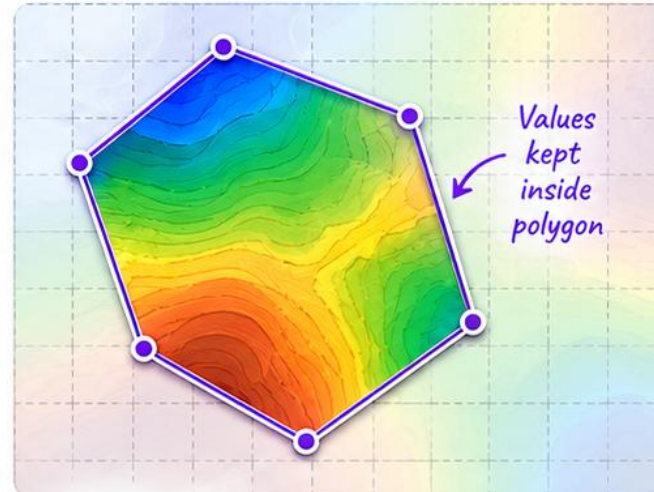
Common uses

- ✓ Subset large rasters to a smaller area
- ✓ Faster analysis
- ✓ Data preparation / clipping



2 Mask

Use a polygon to keep cells inside the shape and set outside cells to NoData.

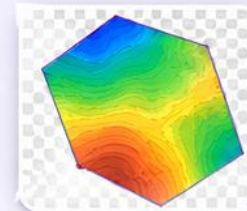


What it does

Keeps raster values inside the polygon; outside becomes NoData.

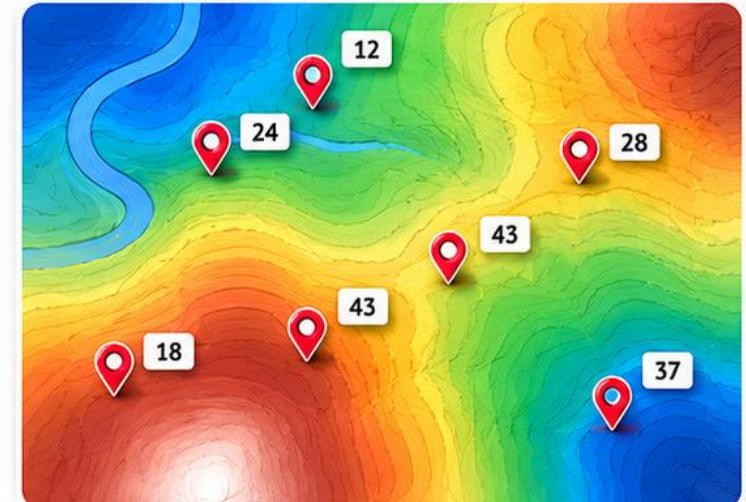
Common uses

- ✓ Clip to administrative boundaries
- ✓ Focus analysis on study area
- ✓ Apply irregular study regions



3 Extract at Points

Retrieve raster values at specific point locations.



What it does

Returns the raster value at each point location.

Common uses

- ✓ Sample elevation, temperature, rainfall
- ✓ Environmental monitoring
- ✓ Model inputs at observation points

Point ID	Value
P1	24
P2	12
P3	35
P4	43
P5	37
P6	18
P7	28

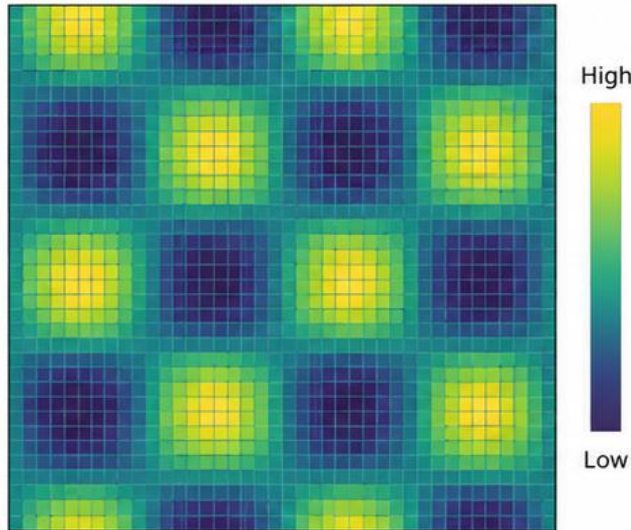
Crop = clip to bounding box | Mask = clip to polygon | Extract = sample at points. These operations bridge the raster and vector worlds.

Resampling Methods — Changing Raster Resolution

Resampling changes the cell size (resolution). Different methods produce different results.

1 Original (40 × 40)

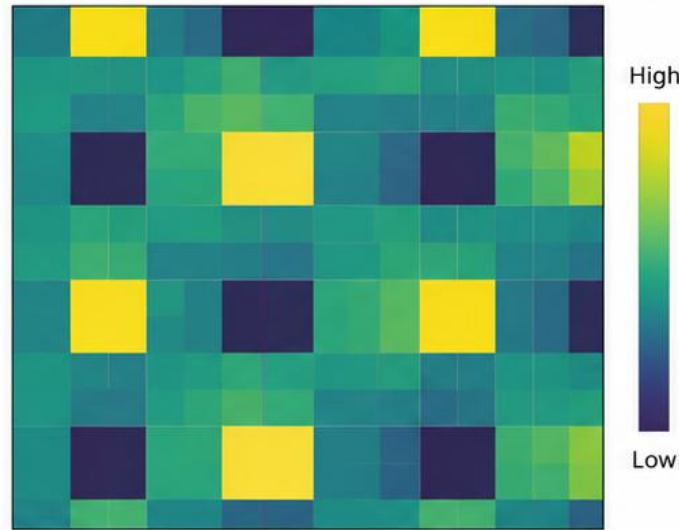
High resolution



Original raster with cell size = 1 unit
(40 × 40 cells)

2 Nearest Neighbour

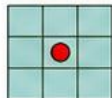
Lower resolution (e.g., 10 × 10)



Blocky appearance — picks the value of the nearest original cell.

Preserves original values (no new values created).

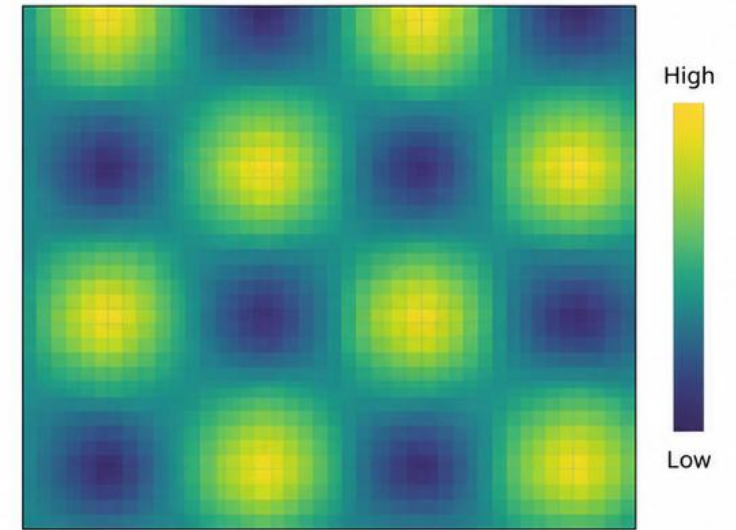
How it works



Assigns the value of the nearest input cell to the output cell.

3 Bilinear Interpolation

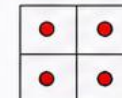
Lower resolution (e.g., 10 × 10)



Smooth appearance — calculates new values using surrounding cells.

Creates new values (interpolated).

How it works



Calculates a weighted average of the four nearest input cells.



Key takeaway: Nearest Neighbour is best for categorical data (preserves values).

Bilinear is best for continuous data (produces smoother results).

Always ask: 'what resampling method was used?'

Raster Math in NumPy

Rasters are just NumPy arrays — use standard array operations:

Boolean mask: which cells are below 3m elevation?

flood_mask = dem <= 3.0

Depth map: how deep is the flood at each cell?

depth = np.where(dem <= 3.0, 3.0 - dem, 0)

Reclassify slope into categories

slope_class = np.digitize(slope, bins=[5, 15, 30])

All vectorised — processes millions of cells in milliseconds.

Computing Area from a Raster

Counting cells and converting to real-world area:

```
# Count inundated cells
```

```
n_flood = int(flood_mask.sum())
```

```
# Cell area = resolution_x * resolution_y
```

```
cell_area = 10 * 10 # 10m DEM -> 100 m2 per cell
```

```
# Total flood area
```

```
flood_area = n_flood * cell_area # in m2
```

```
flood_area_ha = flood_area / 10_000 # convert to hectares
```

This is how engineers estimate flood extents, landslide areas, and habitat loss.

Canterbury Case Study: Slope Stability Assessment

Scenario: assess slope stability risk across the Port Hills

1. Load LINZ 1m DEM of Port Hills area
2. Compute slope (degrees) — focal operation
3. Classify: $>30^\circ$ = high risk, $15-30^\circ$ = moderate, $<15^\circ$ = low
4. Overlay building footprints — which structures are on steep slopes?
5. Extract slope value at each building centroid (point extraction)
6. Generate hillshade map for visualisation and reporting

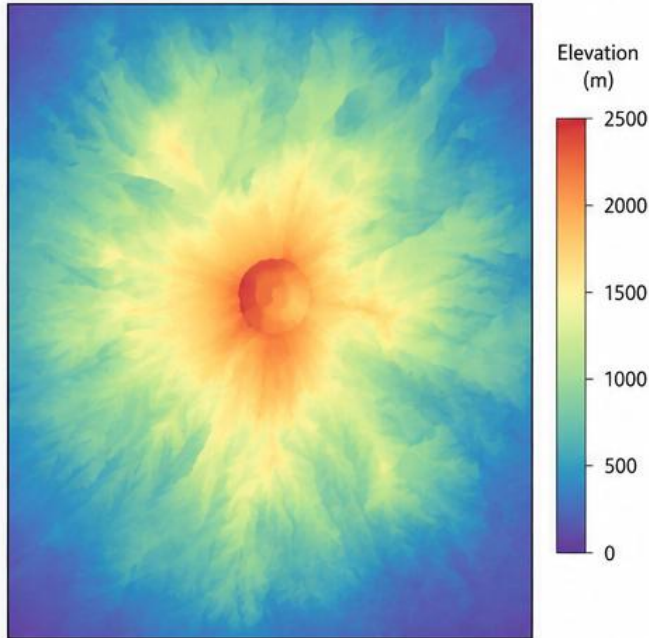
This is exactly what was done after the 2011 earthquakes for rockfall risk.

The Power of Hillshade: Revealing Volcanic Terrain Structure

Hillshade adds lighting and shadows to elevation data, making landforms easier to see and interpret.

1. Elevation (No Hillshade)

Hard to interpret

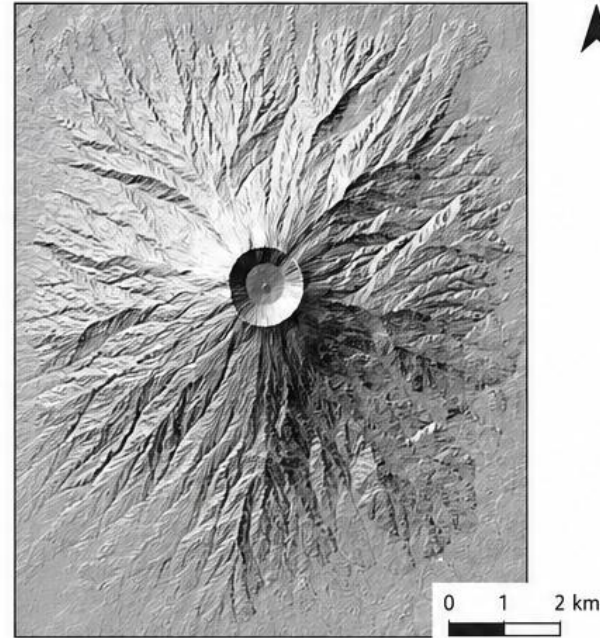


What you see

- Elevation values only
- Hard to distinguish terrain features and landforms

2. Hillshade

Terrain comes to life

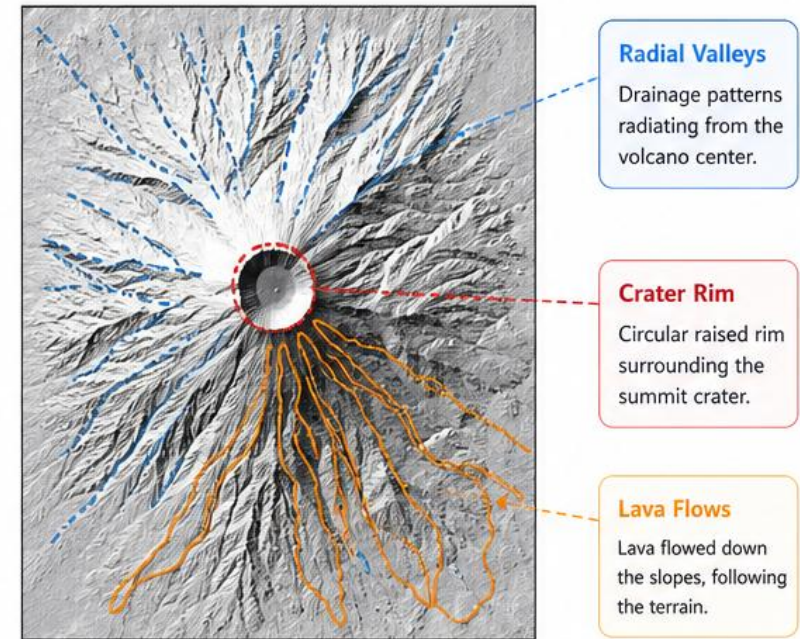


What you see

- Lighting and shadows reveal shape, slopes, and depth
- Landforms become clear and interpretable

3. Interpreted Volcanic Features

Hillshade reveals the story



Radial Valleys

Drainage patterns radiating from the volcano center.

Crater Rim

Circular raised rim surrounding the summit crater.

Lava Flows

Lava flowed down the slopes, following the terrain.



What it reveals

- Volcanic structure and processes
- Erosion and drainage patterns
- Lava flow extent and pathways
- Hazard assessment and planning

Why it matters



Geology & geomorphology



Hazard mapping



Field planning



Better decisions



Key takeaway:

Hillshade transforms elevation data into a clear 3D-like view, making volcanic landforms easy to see, interpret, and understand.

Today's Takeaways

1. Rasters have key properties: resolution, extent, CRS, bands, NoData
2. Map algebra has four levels: local, focal, zonal, global
3. Slope, aspect, and hillshade are focal operations on DEMs
4. Raster-vector interaction: crop, mask, extract connect the two data models
5. Raster area = count of qualifying cells × cell area

Discussion

**A 10 m DEM shows 2,500 cells below a 3 m flood level.
Calculate the flood area in hectares.**

Work through it:

What is the area of one cell?

What is the total flooded area in m²?

Convert to hectares (1 ha = 10,000 m²)

Answer: $10 \times 10 = 100 \text{ m}^2/\text{cell}$. $2,500 \times 100 = 250,000 \text{ m}^2 = 25 \text{ ha}$.

Follow-up: What if the DEM was 1m resolution instead? Would the answer change?